

COMP3320/6464 Week 6, Lecture 1

Rhys Hawkins

Semester 2, 2024

Lecture Plan

This week

- Matrix Transpose
- Matrix Multiply

Matrix Transpose

```
void matrix_transpose(int N, double *restrict A, double *restrict B)
{
    int i;
    int j;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            A[j*N + i] = B[i*N + j];
        }
    }
}
```

- Equivalent to $A[j][i] = B[i][j]$ if arrays were declared as 2D arrays
- Reads are stride 1
- Writes are stride N

Expected Performance

- Read is optimal (Stride 1)
- Overall performance depends on N and on Cache Write Policy
 - Write Through
 - Write Back
- For small N , if both matrices fit into L1/L2 cache, performance should be at the memory bandwidth of the cache speed
- For large N , cache read/write misses need to be taken into account.

Expected Performance : Write Through

What happens during a cache write miss for write through cache?

- Assuming write-allocate: cache line is read
- Updated memory is written back
- On cache line eviction, no writing required.

For matrix transpose

- Write miss is more expensive than read miss
- Cache write misses are nearly twice as expensive as cache read misses (2x memory access)

Expected Performance : Write Back

What happens during a cache write miss for write back cache?

- Write back cache are generally write-allocate (fetch on write): cache line is read
- Updated memory is written to cache only (quick)
- On cache line eviction, cache is written to memory

For matrix transpose

- Write miss is slightly more expensive than read miss
- Memory write will occur when cache line is evicted

Memory Access Optimisation

- Changing the order of the loops
- Generally write misses are more expensive than read misses
- In matrix transpose example: stride N reads are preferable to stride N writes.

```
void matrix_transpose_flipped(int N, double *A, const double *B)
{
    int i;
    int j;

    for (j = 0; j < N; j++) {
        for (i = 0; i < N; i++) {
            A[j*N + i] = B[i*N + j];
        }
    }
}
```

Performance : Intel “Nocoma” (Book)

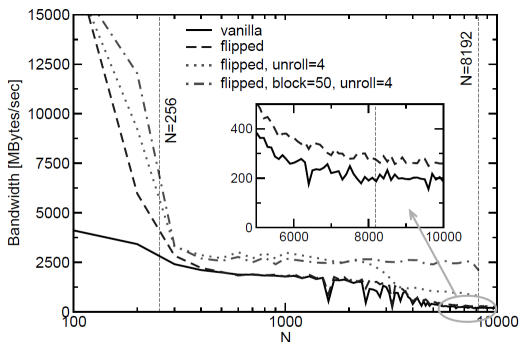
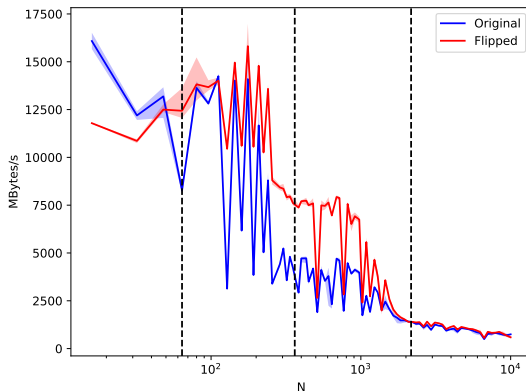


Figure 3.8: Performance (effective bandwidth) for different implementations of the dense matrix transpose on a modern microprocessor with 1 MByte of L2 cache. The $N = 256$ and $N = 8192$ lines indicate the positions where the matrices fully fit into the cache and where N cache lines fit into the cache, respectively. (Intel Xeon/Nocona 3.2 GHz.)

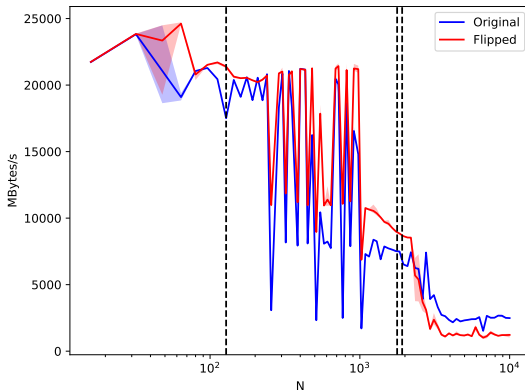
- Intel Nocoma released in 2004
- Flipped slightly faster for large N

Performance : Gadi



- Intel Cascade Lake released 2019
- Flipped better in L2/L3 Caching regime
- Flipped/Original Similar performance for large N

Performance : Apple M1



- Apple M1-Pro Released 2021
- Flipped slower than Original for large N

Summary : Flipping

- Flipping generally improves performance but sometimes it is marginal
- The general idea is that strided writes are more expensive than strided reads
 - Write miss = Memory read + Memory write (Write Through)
 - Write miss = Memory read + Cache write (Write Back)

Flipping and Loop Unrolling

```
int i;
int j;

for (j = 0; j < N; j++) {
    for (i = 0; i < N; i++) {

        A[j*N + i] = B[i*N + j];

    }
}
```

- Load $B[i*N + j]$ means that $B[i*N + j + 1]$ will generally be loaded in cache line
- We can unroll the outer loop to take advantage of this.

Flipping and Loop Unrolling

```
int i;
int j;

for (j = 0; j < N; j += 2) {
    for (i = 0; i < N; i++) {

        A[j*N + i] = B[i*N + j];

    }
    for (i = 0; i < N; i++) {

        A[(j + 1)*N + i] = B[i*N + j + 1];

    }
}
```

- Unroll outer loop
- Has this changed anything?
- The two loop bodies are independent and can be fused.

Flipping and Loop Unrolling

```
int i;
int j;

for (j = 0; j < N; j += 2) {
    for (i = 0; i < N; i++) {

        A[j*N + i] = B[i*N + j];
        A[(j + 1)*N + i] = B[i*N + j + 1];

    }
}
```

- Unrolling outer loops and loop fusion is called loop unroll and jam.
- 2nd read of $B[i*N + j + 1]$ will be from cache (locality)
- Standard worst case was two cache misses per iteration
- Now worst case is 3 misses per two iterations

Performance : Intel “Nocom” (Book)

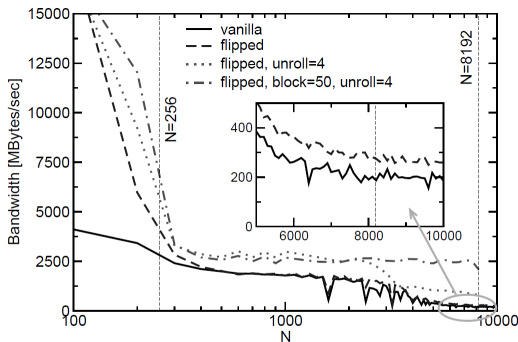
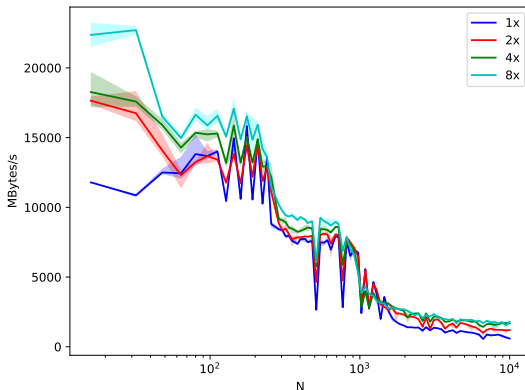


Figure 3.8: Performance (effective bandwidth) for different implementations of the dense matrix transpose on a modern microprocessor with 1 MByte of L2 cache. The $N = 256$ and $N = 8192$ lines indicate the positions where the matrices fully fit into the cache and where N cache lines fit into the cache, respectively. (Intel Xeon/Nocona 3.2 GHz.)

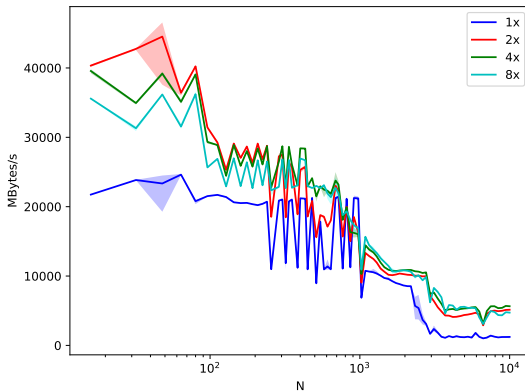
- Unroll x4 gives significant performance improvement at large N

Performance : Gadi



- At large N
 - x2 roughly two times as fast
 - x4 roughly four times as fast
 - x8 similar to x4 performance

Performance : Apple M1



- Similar performance improvement

Loop Unroll and Jam Summary

- The standard flipped version has as a worst case a cache write miss and cache read miss per iteration
- By unrolling the outer loop, we can reduce the number of cache misses on the strided reads
- Loop unroll and jam gives consistent performance improvements
- Most efficient use of cache likely to give best performance overall
- Why does x8 not perform much better than x4?

Blocking algorithms

- A general technique for improving performance
- Use knowledge of L1/L2 cache size to maximise cache use efficiency (e.g. use full cache lines only once)

Matrix Transpose: Blocked Version

```
void
matrix_transpose_flipped_block(int N, double *A, const double *B, int blocksize)
{
    int i, j, ii, jj;

    for (jj = 0; jj < N; jj += blocksize) {
        int jstart = jj;
        int jend = jj + blocksize;

        for (ii = 0; ii < N; ii += blocksize) {
            int istart = ii;
            int iend = ii + blocksize;

            for (j = jstart; j < jend; j++) {

                for (i = istart; i < iend; i++) {
                    A[j*N + i] = B[i*N + j];
                }
            }
        }
    }
}
```

- Assumes blocksize is a multiple of N
- For small blocksize, block of A and B can be stored in L1 or L2 cache.

Performance : Intel “Nocom” (Book)

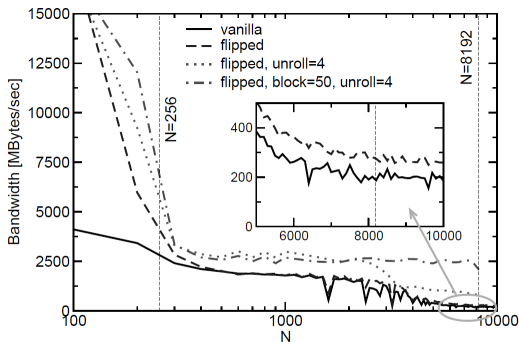
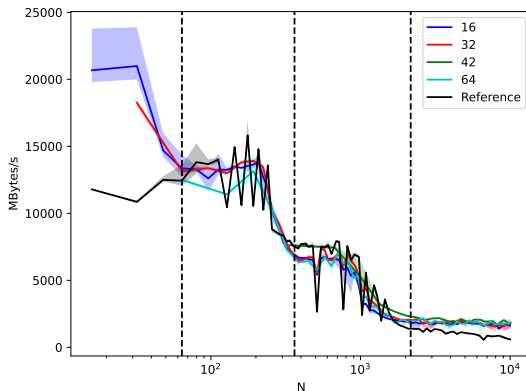


Figure 3.8: Performance (effective bandwidth) for different implementations of the dense matrix transpose on a modern microprocessor with 1 MByte of L2 cache. The $N = 256$ and $N = 8192$ lines indicate the positions where the matrices fully fit into the cache and where N cache lines fit into the cache, respectively. (Intel Xeon/Nocona 3.2 GHz.)

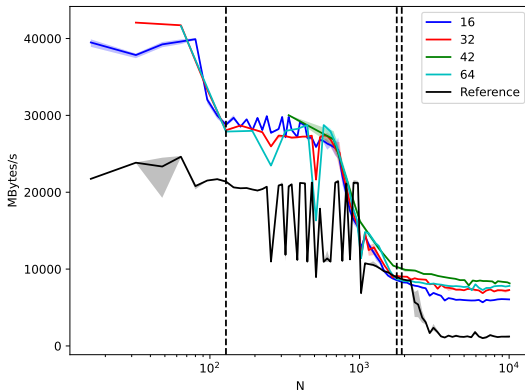
- Blocking gives large performance improvement for large N

Performance : Gadi



- Blocking gives approximately double performance at large N
- Blocksize of 42 is slightly better
- Run on head node

Performance : Apple M1



- Clearer display of improvements due to blocking and effect of blocksize

General approaches for improving memory access

- Matrix transpose is a good example of a difficult memory access pattern
- In general: cache write miss is worse than cache read miss so flipping loop orders to prefer good access patterns for writes
- Use loop unroll and jam (loop fusion) to take advantage of cache lines
- Blocking algorithms aim to use cache efficiently
- The aim is to only load/store each memory address the minimum required for the algorithm (once for matrix transpose)
- All of these strategies behave differently on different CPUs, compilers etc
 - Carefully benchmark to choose unroll level, block size etc